

Untitled-2

```
1
2 # %%
3 import pandas as pd
4 import numpy as np
5 import matplotlib.pyplot as plt
6 from matplotlib.patches import Patch
7 import lightgbm as lgb
8 import tensorflow as tf
9 from tensorflow.keras.models import Model
10 from tensorflow.keras.layers import Input, Conv1D, MaxPooling1D, GRU, Dense, Dropout,
    BatchNormalization, Bidirectional
11 from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
12 from sklearn.preprocessing import MinMaxScaler
13 from sklearn.metrics import mean_squared_error, f1_score, precision_score, recall_score
14
15 n_past = 14
16 threshold = 50
17 LGB_COLOR = '#657af0'
18 CNN_COLOR = '#f57b52'
19
20 # %%
21 # 여수 데이터
22 df1 = pd.read_csv('C:/Users/Urangoo/OneDrive/Desktop/yeosu3/yeosu23.csv', encoding='utf-8-
    sig')
23 df2 = pd.read_csv('C:/Users/Urangoo/OneDrive/Desktop/yeosu3/yeosu24.csv', encoding='utf-8-
    sig')
24 df3 = pd.read_csv('C:/Users/Urangoo/OneDrive/Desktop/yeosu3/yeosu25.csv', encoding='utf-8-
    sig')
25 df = pd.concat([df1, df2, df3], ignore_index=True)
26 df.columns = [c.strip() for c in df.columns]
27 df['일시'] = pd.to_datetime(df['일시'], errors='coerce')
28 df['날짜'] = df['일시'].dt.date
29 df['누적강수량(mm)'] = pd.to_numeric(df['누적강수량(mm)'], errors='coerce')
30 daily_df = df.groupby('날짜')['누적강수량(mm)'].max().reset_index().dropna()
31
32 scaler= MinMaxScaler()
33 scaled_data = scaler.fit_transform(daily_df[['누적강수량(mm)']])
34 dates = daily_df['날짜'].astype(str).values
35
36 X, y = [], []
37 for i in range(n_past, len(scaled_data)):
38     X.append(scaled_data[i-n_past:i].flatten())
39     y.append(scaled_data[i, 0])
40 X = np.array(X)
41 y = np.array(y)
42 dates = dates[n_past:]
43 step = max(1, len(dates) // 25)
44
```

```

45 # %%
46 # LightGBM
47 feature_names = [f'lag_{i+1}' for i in range(n_past)]
48 X_df = pd.DataFrame(X, columns=feature_names)
49
50 lgb_model = lgb.LGBMRegressor(n_estimators=2000,
51                               learning_rate=0.05,
52                               random_state=42,
53                               verbose=-1)
54 lgb_model.fit(X_df, y)
55
56 lgb_pred = scaler.inverse_transform(lgb_model.predict(X_df).reshape(-1, 1))
57 real = scaler.inverse_transform(y.reshape(-1, 1))
58
59 lgb_mse = mean_squared_error(real, lgb_pred)
60 lgb_rmse = np.sqrt(lgb_mse)
61 lgb_mae = float(np.mean(np.abs(real - lgb_pred)))
62 lgb_r2 = float(1 - np.sum((real - lgb_pred)**2) / np.sum((real - np.mean(real))**2))
63 rb = (real >= threshold).astype(int)
64 pb = (lgb_pred >= threshold).astype(int)
65
66 print("LightGBM Yeosu")
67 print(f"MSE: {round(lgb_mse,3)} RMSE: {round(lgb_rmse,3)}")
68 print(f"Precision: {round(precision_score(rb,pb,zero_division=0),3)} "
69       f"Recall: {round(recall_score(rb,pb,zero_division=0),3)} "
70       f"F1: {round(f1_score(rb,pb,zero_division=0),3)}")
71
72 lgb_result = {'MSE': round(lgb_mse,4), 'RMSE': round(lgb_rmse,4),
73              'MAE': round(lgb_mae,4), 'R2': round(lgb_r2,4), 'NSE': round(lgb_r2,4)}
74
75 plt.figure(figsize=(18, 6))
76 plt.plot(dates, lgb_pred, color='tab:orange', lw=1.0, label='LightGBM Prediction', zorder=1,
77          alpha=0.9)
78 plt.plot(dates, real, color='tab:blue', lw=1.5, label='Real Rainfall', zorder=2,
79          alpha=0.8)
80 plt.xticks(np.arange(0, len(dates), step), dates[::step], rotation=45)
81 plt.xlabel('Date'); plt.ylabel('Rainfall (mm)')
82 plt.title('LightGBM Rainfall Prediction [Yeosu]')
83 plt.legend(); plt.grid(alpha=0.3); plt.tight_layout()
84 plt.show()
85
86 # %%
87 # 1D-CNN+GRU
88 scaler= MinMaxScaler()
89 scaled_data = scaler.fit_transform(daily_df[['누적강수량(mm)']])
90
91 X, y = [], []
92 for i in range(n_past, len(scaled_data)):
93     X.append(scaled_data[i-n_past:i].flatten())

```

```

93     y.append(scaled_data[i, 0])
94 X = np.array(X)
95 y = np.array(y)
96 X_seq = X.reshape(X.shape[0], n_past, 1)
97
98 tf.keras.backend.clear_session()
99 inp = Input(shape=(n_past, 1))
100 x = Conv1D(64, kernel_size=3, activation='relu', padding='same')(inp)
101 x = BatchNormalization()(x)
102 x = Conv1D(64, kernel_size=3, activation='relu', padding='same')(x)
103 x = BatchNormalization()(x)
104 x = MaxPooling1D(pool_size=2)(x)
105 x = Dropout(0.2)(x)
106 x = Bidirectional(GRU(64, return_sequences=True))(x)    # return_sequences=True
107 x = Dropout(0.2)(x)
108 x = GRU(32, return_sequences=False)(x)                # нэмэлт давхарга
109 x = Dropout(0.2)(x)
110 x = Dense(32, activation='relu')(x)
111 out = Dense(1, activation='sigmoid')(x)
112
113 model = Model(inp, out)
114 model.compile(optimizer=tf.keras.optimizers.Adam(1e-3), loss='mse')
115
116 model.fit(
117     X_seq, y,
118     epochs=1000,
119     batch_size=8,
120     validation_split=0.2,
121     callbacks=[
122         EarlyStopping(patience=50,
123                        restore_best_weights=True,
124                        verbose=0),
125         ReduceLROnPlateau(patience=20,
126                           factor=0.5,
127                           min_lr=1e-6,
128                           verbose=0),
129     ],
130     verbose=1,
131 )
132
133 cnn_pred = scaler.inverse_transform(model.predict(X_seq, verbose=0))
134 real     = scaler.inverse_transform(y.reshape(-1, 1))
135
136 cnn_mse = mean_squared_error(real, cnn_pred)
137 cnn_rmse = np.sqrt(cnn_mse)
138 cnn_mae = float(np.mean(np.abs(real - cnn_pred)))
139 cnn_r2 = float(1 - np.sum((real - cnn_pred)**2) / np.sum((real - np.mean(real))**2))
140 rb = (real >= threshold).astype(int)
141 pb = (cnn_pred >= threshold).astype(int)
142

```

```

143 print(f"\npred max: {cnn_pred.max():.2f}  real max: {real.max():.2f}")
144 print("1D-CNN+GRU Yeosu")
145 print(f"MSE: {round(cnn_mse,3)}  RMSE: {round(cnn_rmse,3)}")
146 print(f"Precision: {round(precision_score(rb,pb,zero_division=0),3)}  "
147       f"Recall: {round(recall_score(rb,pb,zero_division=0),3)}  "
148       f"F1: {round(f1_score(rb,pb,zero_division=0),3)}")
149
150 cnn_result = {'MSE': round(cnn_mse,4), 'RMSE': round(cnn_rmse,4),
151              'MAE': round(cnn_mae,4), 'R2': round(cnn_r2,4), 'NSE': round(cnn_r2,4)}
152
153 plt.figure(figsize=(18, 6))
154 plt.plot(dates, cnn_pred, color='tab:orange', lw=1.0, label='1D-CNN+GRU Prediction',
155          zorder=1, alpha=0.9)
156 plt.plot(dates, real, color='tab:blue', lw=1.5, label='Real Rainfall',
157          zorder=2, alpha=0.8)
158 plt.xticks(np.arange(0, len(dates), step), dates[::step], rotation=45)
159 plt.xlabel('Date'); plt.ylabel('Rainfall (mm)')
160 plt.title('1D-CNN+GRU Rainfall Prediction [Yeosu]')
161 plt.legend(); plt.grid(alpha=0.3); plt.tight_layout()
162 plt.show()
163
164 # %%
165 # 최종 비교 bar chart
166 summary = pd.DataFrame([lgb_result, cnn_result],
167                        index=['LightGBM [Yeosu]', 'CNN+GRU [Yeosu]'])
168 metrics = ['MSE', 'RMSE', 'MAE', 'R2', 'NSE']
169 colors = [LGB_COLOR, CNN_COLOR]
170
171 fig, axes = plt.subplots(1, len(metrics), figsize=(15, 5))
172 for ax, m in zip(axes, metrics):
173     vals = [summary.loc[mdl, m] for mdl in summary.index]
174     bars = ax.bar(range(len(summary)), vals, color=colors, width=0.6, edgecolor='white')
175     ax.set_xticks(range(len(summary)))
176     ax.set_xticklabels(summary.index, rotation=15, ha='right', fontsize=7)
177     ax.set_title(m, fontweight='bold')
178     for bar, v in zip(bars, vals):
179         ax.text(bar.get_x()+bar.get_width()/2, bar.get_height()*1.01,
180                f'{v:.4f}', ha='center', va='bottom', fontsize=7)
181     ax.grid(axis='y', alpha=0.3)
182 fig.legend(handles=[
183     Patch(facecolor=LGB_COLOR, label='LightGBM'),
184     Patch(facecolor=CNN_COLOR, label='CNN+GRU')],
185 loc='upper center', bbox_to_anchor=(0.5, 1.05), ncol=2, fontsize=10)
186 fig.suptitle('모델 성능 비교 [Yeosu]', fontsize=13, fontweight='bold')
187 plt.tight_layout()
188 plt.show()
189
190 print("\n완료!")
191 # %%

```

